

A Restricted Euclidean TSP Heuristic for Structured Geometric Instances

Abstract

We study a restricted version of the traveling salesman problem in which the input consists of points in the Euclidean plane and the objective is to find a Hamiltonian cycle of minimum total length. The problem is well known in its unrestricted form, but this work does not claim a polynomial-time exact algorithm for the general case. Instead, we focus on a geometrically structured subclass of Euclidean instances for which a transparent heuristic can be defined and evaluated. The proposed method combines a nearest-neighbor construction phase with a local 2-opt refinement step. This design is intentionally conservative: it makes explicit the assumptions on the input, clarifies the scope of the algorithm, and avoids overgeneralization beyond the restricted domain under consideration. The resulting prototype is suitable for empirical evaluation on synthetic geometric instances and serves as a scientifically disciplined foundation for further study of structured Euclidean TSP variants.

1 Problem setting

Let $P = \{p_1, \dots, p_n\} \subset \mathbb{R}^2$ be a set of points. The goal is to find a permutation T of these points such that the tour length

$$L(T) = \sum_{(i,j) \in T} \|p_i - p_j\|_2$$

is minimized. This is the Euclidean TSP, but the algorithm is designed for a restricted geometric setting rather than for arbitrary weighted complete graphs.

2 Motivating reasoning

The central issue is not whether the general TSP can be solved exactly in polynomial time, but rather how a well-posed computational question can

be studied under explicit assumptions. In the unrestricted case, the problem is defined over arbitrary weighted graphs and no geometric regularity can be used to simplify the search. In contrast, when the input is a set of planar points under the Euclidean metric, the geometric structure can be exploited to design constructive and local-search procedures.

This suggests a disciplined progression:

1. specify the restricted domain of interest;
2. define a problem statement compatible with that domain;
3. design a method that uses only the assumptions available in that domain;
4. validate the method empirically without inflating the claims beyond the domain of validity.

The present work follows this logic. It does not attempt to solve the general TSP, but instead studies a realistic geometric subproblem whose assumptions are explicit and whose conclusions remain bounded by those assumptions.

3 Assumptions and scope

The method relies on the following assumptions:

- Euclidean input in the plane;
- complete graph over the points;
- distance defined by the standard Euclidean norm;
- structured or moderately regular geometric instances;
- focus on practical solution quality rather than general exactness.

Under these assumptions, the method is meaningful as a prototype for a restricted subclass of TSP instances. It does not claim a general polynomial-time exact algorithm for the unrestricted TSP.

4 Step 1: restriction of the problem

The first design choice is to restrict the input class. Instead of allowing arbitrary weights or arbitrary graph structure, we assume that cities are points in the Euclidean plane. This makes the problem conceptually simpler and also computationally more structured. It allows the algorithm to rely on geometric proximity, which is not available in the general weighted case.

This restriction is important for scientific correctness. In a paper or technical report, a method should be stated precisely in terms of the class of instances it addresses. Otherwise, a heuristic designed for Euclidean data may be mistakenly interpreted as a result on the unrestricted TSP.

5 Step 2: constructive heuristic

The next step is to define a constructive rule. A natural choice is a nearest-neighbor strategy: starting from an arbitrary point, the algorithm repeatedly appends the unvisited point closest to the current one. This produces a valid Hamiltonian cycle and is computationally inexpensive.

The rationale is straightforward. If the points are geometrically well behaved, then local proximity tends to provide a useful starting structure. Such a construction is not intended to be optimal in every case; it is intended to create an initial feasible tour that can then be refined.

6 Step 3: local refinement

The constructive phase is followed by a local optimization step. The 2-opt heuristic checks whether reversing a segment of the tour reduces the total travel length. If so, the reversal is accepted. The procedure is repeated until no further improvement is possible.

This step is essential because it transforms a merely feasible tour into a more competitive one. It also makes the algorithm more realistic from an optimization perspective: a strong initial tour is often improved substantially by local search, even when global optimality is not guaranteed.

7 Step 4: empirical interpretation

The method is best understood as an empirical prototype rather than as a theorem-level solution. Its value is not in proving universal optimality, but in showing that a restricted geometric instance class can be handled with a

disciplined algorithmic design. Measurements such as tour length, runtime, and convergence behavior are then meaningful because they describe the performance of the method under explicit assumptions.

For this reason, the algorithm should be evaluated on synthetic Euclidean inputs with different structure: random points, clustered points, and grid-like arrangements. Such experiments help identify when the heuristic is effective and when the geometric assumptions are too weak or the instance class too irregular.

8 Method

The procedure has two phases. First, it builds a feasible tour using a nearest-neighbor rule: starting from an arbitrary point, it repeatedly selects the unvisited point closest to the current one. Second, it performs local improvement via 2-opt, reversing segments when the total tour length decreases.

Algorithm 1 Restricted Euclidean TSP heuristic

Require: Points $P = \{p_1, \dots, p_n\} \subset \mathbb{R}^2$

Ensure: A Hamiltonian cycle T

```
1:  $S \leftarrow P$ 
2: choose initial point  $s \in S$ 
3:  $T \leftarrow [s]$ 
4: while  $S \setminus \{s\} \neq \emptyset$  do
5:    $v \leftarrow \arg \min_{u \in S \setminus \{s\}} \|u - s\|_2$ 
6:   append  $v$  to  $T$ 
7:    $s \leftarrow v$ 
8: end while
9: append the first point to close the cycle
10: repeat
11:   improved  $\leftarrow$  false
12:   for all pairs of edges  $(i, j)$  and  $(k, l)$  in  $T$  do
13:     if reversing the segment between  $j$  and  $k$  decreases the total
        length then
14:       reverse that segment
15:       improved  $\leftarrow$  true
16:     end if
17:   end for
18: until not improved
19: return  $T$ 
```

9 Discussion

This heuristic is transparent, easy to implement, and naturally suited to benchmark-based experimentation. However, it does not provide optimality guarantees for arbitrary weighted graphs or for all Euclidean instances. Its scientific value lies in its explicit restriction to a problem class and in the honesty of its assumptions. The method is strongest when interpreted as a structured geometric prototype, not as a universal solver for the unrestricted TSP.

10 Evidence-driven literature refinement

The accumulated corpus contains 137 records with a coverage score of 0.905. Under the revised stopping criterion, the corpus contains 0 claims matching

the explicit unrestricted deterministic polynomial-time exact requirements and 0 independently verified algorithms. The previous “objective reached” status reflected bibliographic coverage, not discovery of the requested algorithm, and is therefore superseded.

No record currently establishes that a deterministic polynomial-time exact algorithm solves unrestricted TSP. Hardness results, exponential exact algorithms, approximations, and special-case algorithms do not satisfy this objective. The search remains inconclusive; reaching the iteration limit or exhausting queries does not count as finding an algorithm.

11 Conclusion

The method provides a coherent prototype for a restricted Euclidean TSP setting. It is best understood as an experimentally testable heuristic for a structured domain, not as a universal exact solution to the general TSP. The overall reasoning is therefore disciplined: the problem is restricted, the assumptions are explicit, the algorithm follows from those assumptions, and the conclusions remain within the intended scope of the model. The evidence-driven literature cycle adds a further layer of rigor by improving the bibliographic coverage and by distinguishing more clearly between exact, approximate, and hardness-oriented results. This makes the contribution suitable as a scientific draft for a special-case optimization study, while preserving a rigorous distinction between restricted solvability and unrestricted hardness.